

C++

solvrx

2026

1. Practice I

- (a) Links (Pointers++)
- (b) Classes and Structures
- (c) C++ is a fool I
- (d) Functions Overload

2. Practice II

- (a) new And delete operators
- (b) Overload of other operators

3. Practice III

- (a) Casts
- (b) Exceptions
- (c) memory control (RAII)

4. Practice IV

- (a) Multi Inheritance
- (b) Method Hiding
- (c) Dispetcher
- (d) Explicit call of constructor
- (e) Slice-effect

5. [Practice V](#)

- (a) [Libraries](#)
- (b) [Exceptions in IOs](#)

6. [Practice VI](#)

- (a) [Templates](#)
- (b) [Some features](#)
- (c) [Specialization](#)
- (d) [For HW](#)

7. [Practice VII](#)

- (a) [For Lab 11](#)
- (b) [For Lab 12](#)
- (c) [Template tricks and notes](#)
- (d) [Fold expressions](#)
- (e) [Static Polymorphysm](#)
- (f) [constexpr](#)

8. [Practice VIII](#)

9. [Practice IX](#)

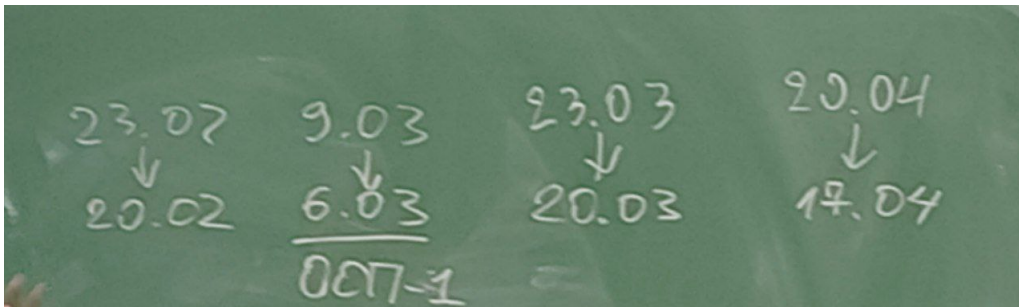
10. [Practice X](#)

11. [Practice XI](#)

12. [Practice XII](#)

13. [Practice XIV](#)

1 Practice I



'Code Golf'

1.1 Links (Pointer++)

Links

`int&ri;`

`*ri(operator)`

`int&ri; //ERR`

`-NO`

`int&&(rvalue)`

(move – semantic)

Pointer

`int * pi;`

`*pi → int`

`int * pi; //(notinitialized)`

`pointers'arithmetic(pi + 1)`

`int * *(pointeronpointer)`

Def.: Ссылка – указатель, больше похожий на переменную

```
int x;
```

```
int& ri = x;
```

```
int *pi = &x;
```

```
// sizeof(T&) == sizeof(T*) == true
```

```
void foo( int& x )
```

```
{
```

```
    x = 10;
```

```
}
```

```
int x = 5;
```

```
foo(x);
```

```
// x == 10
```

Note: Link from pointer and visa-versa:

```
int x;
int* px = &x;
int&rx = *px;
int&rx2 = x;

int& foo()
{
    int x = 5;
    return x; // UB
}

// -Wall -Wextra -pedantic : returns warning

int& foo()
{
    int* x = new int[10];
    int& r= x[5];
    delete [] x;
    return r; // Ambiguous Behaviour (AB)
}
```

1.2 Classes and Structures

```
// C

typedef struct str_name_s
{
    str_name_t;
}

// sizeof(empty_struct) == 0

// C++
```

```
struct str_name_t
{

};
```

```
// sizeof(empty_struct) == 1 == sizeof(empty_class)
```

Note: In C++ can refer to a structure only by name:

```
str_name_t x;
```

```
class cls_name_t
{
    ...
};
```

Difference between classes and structures:

Struct – public;

Class – private

Classes syntax:

```
class CLS
{
    private:
    protected:
    public:
        int x;
        int foo();
        static void bar();
};
```

```
CLS::bar() // CLS.bar() in C#
```

Note: classes in C++ can't be Private/Protected if they're in global space, otherwise – yap.

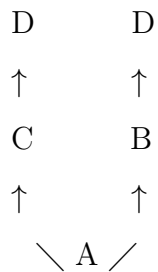
Inheritance:

```
class CLS: public Parent, protected PParent, private PriParent
{
    ...
};
```

// inheritance could be multiple

virtual Inheritance:

```
class CLS: virtual VP
{
    ...
};
```



After virtual D:

Factorize D.

Interface:

```
class CLS : public Parent, IIFace
{
    // interface
    struct IIFace
    {
        virtual int Method1() = 0;
        virtual int Method2(int x) = 0;
    };
}
```

// analogue to abstract

Constructors and Destructors:

```

class Cls
{
    Cls() = default;
    Cls() = delete; // can't define a class with default constr

    Cls(const Cls& other); // copying constructor, pointer won't work
    // = default;

    Cls(Cls&& other); // = default;
    // move-semantic constructor

    ~Cls(); // default destructor

    // if there're some memory in class, wipe it manually
    // through destructor's implementation
}

```

RAII – Resource Acquisition Is Initialization: binds the lifetime of resources to the lifetime of an object (e.g. add destructors after scope)

1.3 C++ is a fool I

Rule: The most vexing parse

```

class Test
{
    Test(int);
    Test(const Test&);
};

Test t1(10);
Test t2(Test(5)); // error further, as it is a func-prototype

// explanation

```

```
// void foo( Test x = 5); -> foo( Test x(5) ) -> foo( Test (5) )
```

```
// instead, use:
```

```
Test t2 = {  
    Test {5}  
};
```

```
Test t3 = Test(50);
```

1.4 Functions Overload

```
void foo( int x ) { }  
void foo( short x ) { }  
void foo( bool x ) { }  
// void foo( const char* x ) { }  
void foo( double x );
```

```
void main()  
{  
    short s = 123;  
    foo(s); // 2  
    foo(s + 5); // 1  
    foo(true); // 3  
    foo("oops!"); // 3, no warnings :)  
    foo(5); // 1  
    foo(5.0); // 5  
  
    foo( cond ? 3 : 3.0 ); // int in any case  
    // (cause uses the first type metnioned, according to  
    // ternar operator rules
```



```
}
```

2 Practice II

Note: `sizeof(malloc(1))` на куче – 16 байт

Note: `p` не кратно 4, тогда нарушение выравнивания с `uint32`

```
char* p;  
*((uint32)*p) = 123;  
  
// better:  
memcpy(p, &x, sizeof(uint32_t))
```

2.1 new And delete operators

Operations:

new delete

new[] delete[]

new implementation:

```
void* operator new(size_t size)  
{  
    return malloc(size);  
}
```

//ex, where new isnt actually an operator:

```
Matrix* m = new Matrix{};
```

$$\left[\begin{array}{l} Matrix * m = operatornew(sizeof(Matrix)); \\ m \rightarrow Matrix(); \end{array} \right.$$

Бывают разные виды new и delete, компилятор ищет нужную сигнатуру

```
// data_ = new char [ sizeof(Matrix)*10]  
void vector::push_back(const Matrix& v)  
{
```

```

        new &data[size_] Matrix{v};
        ++size_;
    }

```

Размещающий оператор new (возвращает просто указатель) + копирующий конструктор

new operator signature:

```
void* operator new(size_t size, std::nothrow) { }
```

```
void* operator new(size_t size, void *ptr) { }
```

Около $6+6 = 12$ new и delete

Как переопределять их для классов?

```

class Matrix
{

    // important incapsulation ,
    // we write 2 new and deletes (for class and array)
    void* operator new(size_t size);
    void operator delete(void *ptr);
};

```

2.2 Overload of other operators

Operators, that can be overloaded:

`= , += -= *= /= %= &= |= ≐`

`«= »= ++x x++ --x x--`

`[] () -> *p &p`

`ClassA a;`

`ClassB b;`

`// folowwing wouldn't compile:`

`a = (T)b;`

```

a = b;

operator Type() const;
explicit operator Type() const;
implicit compiles both, explicit only first

== != < > <= >= <=>

+ - * / %

побитовые операции
&& || !

// exists: std::ratio<1,100>

class Ratio
{
public:
    Ratio(int n, int d); // 1
    ~Ratio() = default;
    Ratio( const Ratio& ) = default; // 2
    Ratio( Ratio&& ) = default; // moving constr (3)
    Ratio& operator = (const Ratio& ) = default; // 4
    Ratio& operator = (Ratio&&) = default; //5

    //rule of 5

    Ratio operator+( const Ratio& other )
    {
        return Ratio{this->n*other.d + this->d->other.n,
                     other.d*this->d}
    }

    int getN() { return this->n; }
    int getD() { return this->d; }
private:
    int n, d;

```

```

}

Ratio& operator+( const Ratio& a, const Ratio& b);
{
    return Ratio{a.getN()*b.getN(),
                 a.getD()*b.getD()}
}

// operator , implementation:
Ratio operator ,(int a, int b)
{
    return Ration{a, b};
}

```

Rule Of 5: Default Constructor exists – we don't need to write else of 5, otherwise – must write

Note: friend (function) could be used for private and protected elements of c other classes (aka internal method)

Note: 'вы с этими запятыми повеситесь, если полезете в libev'

3 Practice III

3.1 Casts

```

static_cast<T>
(T) x

```

Note: (T) x could be UB if u're trying to push more B in less B

```

dynamic_cast<T>

```

T*

T&

```

Base *b = ?;

```

```

Child1* c = dynamic_cast<Child1*>(b);

```

```

Child1 &r = dynamic_cast<Child1&>(b); //std::bad_cast if smth bad hpnd

```

Note: operators is, as $\diamond$ could be implemented with casts

RTTI: run time type information

```
const_cast<T>
```

```
– const
```

```
– volatile
```

```
// ex.:
```

```
std::string::c_str() -> const char*
```

```
// might be UB and violate system
```

```
reinterpret_cast<T>
```

3.2 Exceptions

```
try
```

```
{
```

```
    code;
```

```
    throw std::runtime_error("Oops!");
```

```
    throw 42; // cant be done on cs
```

```
    throw false;
```

```
    throw "string";
```

```
}
```

```
catch( int e )
```

```
{ }
```

```
catch( std::exception e ) // every excep, inherited by e
```

```
// in stack trace overrun it will crash dump exe, so better
```

```
// to do instead:
```

```
catch( std::exception& e)
```

```
{
```

```
    throw; // throws curr except higher
```

```
    throw e;
```

```
}
```

Note: in C++ could be thrown literally ANYTHING

Note: `std::exception`

Note: no finally, so make it yourself)

Note: $\geq$ C++11:

```
void foo() noexcept
```

```
{
```

```
    throw 1; // std::terminate
```

```
}
```

```
// noexcept == noexcept(true) != noexcept(false)
```

Note: all destructors have (undeletable) `noexcept` method:

```
~AllDtors() noexcept
```

```
void foo() throw(A, int)
```

```
    throw()
```

```
// catches any type of exc-ns
```

```
catch(...)
```

```
{
```

```
}
```

```
// but its work may differ due to architecture
```

Note (for Win): SEH exceptions require special libraries (for example `core-dump`)

3.3 memory control (RAII)

Note: "undirected work with pointers"

`<memory>`

1. `unique_ptr<T>`

Def.: smart pointer, can't be copied and exists only one through all program

2. `shared_ptr<T>` / `weak_ptr`

Def.: on object has a lot of pointers on him, but when there is not a single one, then object deletes.

Def.: `weak_ptr` doesn't hold pointers' counter.

3. `auto_ptr`

Note: u can combine these to use even less memory with:

`std::vector`

`std::array`

`std::map`

Note: `std::make_shared<T>()` – creates `shared_ptr` automatically

4 Practice IV

4.1 Multi Inheritance

Note: Try not to use multi-virtual parents to avoid problems with duplicating children. But yes, if you're mentioning class A : virtual B u factorize children. Inherit only one class, other should be interfaces.

```
class Base
{
public:
    virtual void foo();
};

class Child : public Base
{
public:
    void foo();
}
```

```

    ~Child() override = default;
};

// compiles good, foo is automatically virtual and overridden (before C++11)
// after — u must mention override for 'override'
// mention final for state that it's the last overridden of a virt func
//

```

4.2 Method hiding

```

struct Base
{
    virtual ~Base() = default;
    virtual void foo();
};

struct Child : Base
{
    ~Child() override = default;
    void foo() override;
};

```

```

Base* b1 = new Base();
Base* b2 = new Child();

```

```

delete b1; // base destr
delete b2; // child destr, then base

```

```

// if not to mention word "virtual" but override,
// it will not be called

```

```

(A::b1) -> foo();

```


4.3 Dispatcher

```
class A
{
public:
    void func(int);
    void func(double);
};
```

```
A* ptr = new A();
void (A::*int_fptr)(int) = &A::func;
void (A::*dbl_fptr)(double) = &A::func;
(ptr->*int_fptr)(123);
```

Note: can be shortened:

```
decltype(&A::func1) fptr;
```

Def.: defines a type

Ex.:

```
void foo(short x, short y)
{
    decltype(x) z = x; // d...e == short
    decltype(x + y) z2 = y; // d...e == int

    // there would be enough to use 'auto'
}
```

Note: decltype is a 'pro-auto' to determine a type of a function return, in a pattern.

Ex.:

```
template <class T1, class T2>

// cant write 'auto' or
// decltype in such scope, so we use:

auto sum(T1 x, T2 y) -> decltype(x + y)
```

```
{  
  
}
```

4.4 Explicit call of constructor

```
Child::Child() : Base(1), _x (123)  
{  
    ...  
}
```

// constructor shortening:

```
Child::Child() : Child(2)  
{ }
```

```
Child::Child(int length)  
{  
    ...  
}
```

Note: avoid using default value-parameters!!!

When different dlls meet together there appears a conflict.

Better to overload :)

Ex.:

```
void foo( int x = 0);
```

```
void foo();
```

```
foo(); /// new compilers throws an error
```

4.5 Slice-effect

!!! IMPORTANT FOR HIRE !!!

Have classes A, B, polymorphism and so on

```
Child c;
```

```
void foo(Base &b)
{
    b.foo();
}
```

```
foo(c);
```

```
// without & — b method
// with — c method
```

5 Practice V

5.1 Libraries

```
#include <iostream>
    <fstream>
    <stringstream>
```

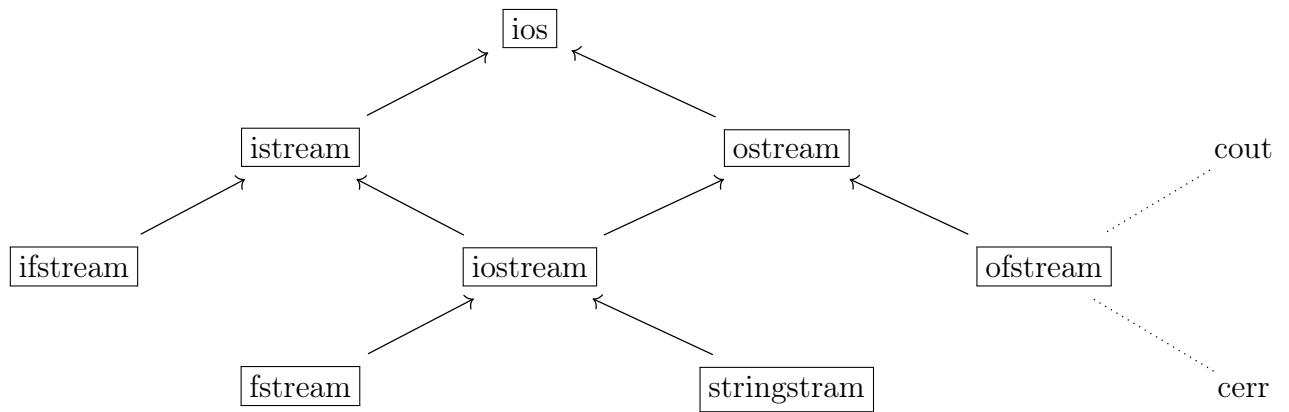
Ex.:

```
printf("%08x", x);
```

```
std::cout << std::hex << std::setw(8)
    << std::setfill('0')
    << x << std::endl;
```

<**format**> looks like `fmt::format("%u")`, might be warning on different platform (unsigned long = unsigned long long)

Note: all IO's inherited with multi-inheriting from IOS base class:



```
std::fstream f("input.txt", std::ios::in);
uint32 u32;
std::string s;
```

```
f >> u32 >> s;
// _\t__\nTEXT
// in u32 — 1
// in s — "TEXT"
```

```
std::ostream& operator <<(std::ostream& s, T x);
// T — type
```

```
std::istream& operator >>(std::istream s, T& x);
```

```
std::cout << "Hi" << std::endl;
// endl — manipulator
```

Note: `std::endl` $\Leftrightarrow$ `System.lineSeparator()` in Java

```
s >> x;
std::getline();
```

```
// reads line, returns s
```

Note: C++ compiler + RAII automatically do `f.close()`;

We can:

```
f.close();
```

```
f.open("f.txt", std::ios::out)
```

Flags:

- in – read
- out – write
- app – add to the end
- binary – binary mode
- trunc – clear file while opening
- ate – stand in the end while opening

Note: to mention many flags use OR (pipe):

```
std::ios::in | std::ios::out | std::ios::binary
```

```
ostream& hex(ostream& os)
```

```
{  
    os.setf(std::ios::hex, std::ios::basefield);  
  
    return os;  
}
```

```
ostream& operator <<(ostream& os, ostream& (*pf)(ostream&))
```

```
{  
    return pf(os);  
}
```

```
os << hex;
```

5.2 Exceptions in IOs

```
f.exceptions(std::ios::failbit | std::ios::badbit |
             std::ios::eofbit);
```

TO catch: `std::ios_base::failure&`

6 Practice VI

6.1 Templates

Note: Templates in C++ – Total Turing Machine (TTM)

```
std::vector<int> iv;
std::vector<double> dv;
```

```
// instantiation points
```

Note: Every templates results in copying identical code blocks. Ex.: Template with 100 * in vector creates 100 copies.

Ex:

- ~ 5 mln. code lines, old C++ – 45 min build time (not to much templates)
- ~ 450 thousands code lines, new C++ – 50 min build time (alot of templates)

Ex.: Templates for multi-platforming (T as platform name in personal lib)

Ex. of template function (the simplest one):

```
template <typename T>
T max( T a, T b )
{
    return (a > b) ? a : b;
}
```

```
auto x = max(1, 2); // instantiation place, but no <>?
```

```
// ans: both are int -> nice
```

```
auto x = max(1, 2.0); // compile error
```

```
// can not instance such
```

```
auto y = max<int>(1, 2.0); // good — we state type in <>
```

What happens with template's func's parameters?

- $x\& \rightarrow x\&$
- $x\&\& \rightarrow x\&\&$
- $x \rightarrow x\&\&$

Def.: $x\&\&$ – R-value: not a v-value, if an object has not a declared name (seen in some scope)

```
struct rtaq
{
    static int x;
};
```

```
struct rtag
{
    int& x;
};
```

```
// they are both types
```

```
max<decltype<T.x>>
// -> int OR int&
```

```
ttt<ctag>
ttt<rtag>
```

```
typenameT::x _x;
// stating a needable type
```

Ex.:

```
struct S1
{
    static int X;
};

struct S2
{
    typedef std::shared_ptr<S2> X;
};

// typename equivalent to class in such context
template <class T>
void foo()
{
    typename T::X x;
    ...
}

foo<S1>(); // compile error, X is not a type
foo<S2>(); // nice
```

6.2 Some features

Note: Starting from C++1720 – auto print out concept

Def.: Concept – definition of a type (may be with some restrictions or requirements)

```
template <auto T>
void print_const()
{
    std::cout << T << std::endl;
}

// similiar to interface of a type
```


Note: Similar to CContract, according to Barbara-Liskov

Note: typename and classes don't add restrictions (better not to use auto)

```
template <class T, class U>
T&& max(T&& a, U&& b)
{
    return (a > b) ? a : b;
}
```

```
// now we can state int + double
// cause we don't fix type for b
```

```
template <class T, class U>
auto add(T a, U b) -> decltype(a + b)
{
    return a + b;
}
```

```
// after C++14:
template <class T, class U>
auto add(T a, U b)
{
    return a + b;
}
```

6.3 Specialization

Def.: Realization of method for concrete type is called Specialization

Part:

```
template <>
Matrix max<Matrix>( Matrix a, Matrix b)
{
    if (a.det() > b.det())
        return a;
}
```

```

        return b;
}

```

6.4 For HW

```

template <class T, int S>
struct array
{
    T data[S];
};

```

```

array<int , 5> = x;
//equiv to:
int x[5];

```

```

template <class T, int S>
struct vector
{
    T* data;

    vector()
    {
        data = new T[S];
    }
};

```

```

// both will lay on heap

```

Note: proxy – operator wiht [], for read/write: from indexator – proxy object

7 Praticice VII

7.1 For 11 Lab

```
#include <type_traits>

static_assert(std::is_default_constructible_v<s1> == true);

struct s1
{
    s1(int){ } //will stop compilation
    std::string str;
};
```

Is typename necessary?

```
typename T::x *y;

// without it compiler thinks that we
// do x * y instead
```

7.2 For 12 Lab

Class

-x
-y
-z

Correct state: everything exists and satisfies the properties. If there's exception – it's in a middle, some parts of a state stays unchanged

Basic guarantee: if there's exception, state can be changed, but after all process – it

Strong guarantee: if there's a problem, state doesn't change.

No throw guarantee: The function is guaranteed never to throw an exception or allow one to propagate. If an exception is thrown internally, it must be caught and handled within the function's implementation. This guarantee is expected of destructors, move constructors,

and swap functions to ensure safe stack unwinding and robust behavior. In modern C++, this is enforced using the `noexcept` specifier.

No exception guarantee: No guarantees are made about the program's state if an exception is thrown. This may lead to resource leaks, corrupted objects, or an unstable program state, and is generally considered a defect to be avoided in well-designed code.

7.3 Template tricks and notes

swap-trick:

```
T& T::operator=(T x)
{
    std::swap(this, &x);
    return this;
}
```

```
void print() {
    std::cout << std::endl;
}
```

```
template <class T class... Args>
void print(first, Args... args) {
    std::cout << first << " ";
    print(args...);
}
```

```
print(1, "Hi", true, 1.23); // prints everything with formatting
```

Note: From <C++14/11:

(beginning of fold-expressions):

```
template <class... Args>
void printAllLegacy(Args... args)
{
    int dummy[] = {0, (std::cout << args << " " << 0)...};
}
```

```

    (void)dummy;
}

```

Result:

```

printAllLegacy(1, "A");

// 1
int dummy[] = {
    0,
    (std::cout << 1 << " ", 0),
    (std::cout << "A" << " ", 0)
};

// 2
{ 0, 0, 0 }; // couts do their work cause of "," operator

//3
(void)dummy; // deletes warning

//4 DONE!

```

7.4 Fold-expressions

C++17+

```

template <class ... Args>
//unary from right
auto sumRight(Args... args)
{
    return (args + ...); // (arg1+(arg2+(arg3 + ...)))
}

// unary from left
auto sumLeft(Args... args)
{

```

```

        return (... + args); // (((arg1+arg2)+arg3)+...)
    }

// unary init
auto sumWithInit(Args... args)
{
    return (0 + ... + args); // ((( 0 + arg1) + arg2) + ...)
}

template <class... Args>
bool allTime( Args... args)
{
    return (true &&...&&args); // (true && arg1 && arg2 &&...);
}

```

7.5 Static Polymorphysm

Def.: CRTP (Curiosity Recurring Template Pattern

Polymorph call without virtual table:

```

template <class Derived>
class Shape
{
public:
    void draw() const
    {
        static_cast<const Derived*>(this)->drawImpl();
    }
};

class Circle : public Shape<Circle>
{
public:
    ...
}

```

```

    void drawImpl() const
    {
        std::cout << "Circle" << std::endl;
    }
};

```

```

class Square : public Shape<Square>
{
public:
    ...
    void drawImpl() const
    {
        std::cout << "Square" << std::endl;
    }
}

```

```

template <class T>
void drawShape(const Shape<T>& s)
{
    s.draw();
}

```

7.6 constexpr

All specializations in one function:

```

template <class T>
auto toString(const T& value)
{
    if constexpr (std::is_arithmetic_v<T>)
    {
        return std::to_string(value);
    } else if constexpr (std::is_same_v<T, std::string>)
    {

```

```

        return value;
    } else
    {
        return std::string("Not supported");
    }
}

```

Def.: constexpr as ifdef in C, but for templates. Does it on instancing step (compilation)

8 Practice VIII

8.1 SFINAE

Def.: Substitution Failure Is Not An Error

```
#include <type_traits>
```

```

template <class T>
typename std::enable_if<std::is_integral<T>::value, T>::type
process( T value)
{
    std::cout << "Integral: " << value << std::endl;
    return value;
}

```

```

template <int N>
struct Factorial
{
    static constexpr int value =
        N * Factorial<N - 1>::value;
};
template <>
struct Factorial<0>

```



```
{
    static constexpr int value = 1;
};
```

Factorial<5>::value //-> factorial(5), compiles on compile time))

8.2 move-semantic

Copy-elision: RVO/NRVO

```
std::vector<int> foo()
{
    std::vector<int> v(100000);
    return v;
}
```

Note: thats bad(

lvalue

```
int x = a;
```

```
int* p = &x; // = &5
```

rvalue (prvalue)

Def.: std::move – static_cast<T&&> (T&x); (changes type of an object r-l value)

```
class point
{
    int* data;
    int data_size;

    point(const point& other);
    point(point&& other);

    point& operator=(const point& o);
    point& operator=(point&& o);
};
```

```

point::point (point&& o)
{
    data = o.data;
    data_size = o.data_size;
    o.data = nullptr;
    o.data_size = 0;
}

```

Note: const restricts rvalue

```

std::vector<int> foo()
{
    std::vector<int> v;

    return std::move(v); // T&& and do not need to type return
}

```

Note: even if theres no copy constr, compiler would always build up a copy-elision constr.

so use std::move

```

std::string s = "Hello ";
std::string t = std::move(s); // s deletes with destr, but s as name exists
std::cout << s;
s = "Hi ";

```

```

const std::string s = "const ";
std::string = std::move(s); // copy-elision

```

```

std::string s = std::move(std::string{"H"});

```

```

for (auto&& x : v) // OK
{
    std::string&& s : v; //ERROR
    int&& x = 5; // OK
}

```

```
std::string x = std::move(v[1]); // OK
```

Note: do not move const!

```
class point
{
    void foo() &;
    void foo() const;
    void foo() &&;
    void foo() const &&;
};
```

```
point p;
std::move(p).foo(); // OK
point{1,2}.foo(); // OK
```

9 Practice IX

9.1 Huffman

AABCAACCCCB

std::priority_queue:
 $\binom{A}{4} \binom{B}{2} \binom{C}{5} - \text{nodes}$
Huffman-Zip

To point (competition in):

- speed
- size
- meta-inf saved size

10 Practice X

10.1 lambdas

```
auto x = [&](params) -> type { body }
```

Def.: Functor – OOP pattern, where object is a function

Note: added in C++11

```
int count = 0;
```

```
auto x = [count]() mutable -> int { return ++count; }
```

```
x(); // 1
```

```
x(); // 2
```

```
// count == 0
```

```
// &count == 2
```

To use with move-semantics:

```
std::vector<int> v{ 1000, 1 };
```

```
auto x = [v = std::move(v)]() -> void {  
    foo(std::move(v));  
} // wont compile, so =>
```

```
auto x = [v = std::move(v)]() mutable -> void {  
    foo(std::move(v));  
}
```

```
auto generic = [](auto a, auto b) { return a + b; }
```

```
int x = generic(1, 2);
```

```
double d = generic(1.2, 3.4);
```

```
std::string s = generic(std::string{"abc"}, std::string{"1"});
```

Note: in C++17 – templates for lambdas added:

```
auto tpl = []<typename T>( T a, T b) -> T {
    return a + b;
};
```

Note: this auto&& self – link to a lambda

```
auto factorial = [](this auto&& self, int n) -> int {
    return (n <= 1) ? 1 : n * self(n - 1);
};
```

```
int (*fptr)(int) = x;
```

```
x(5);
fptr(7);
```

10.2 std::forward

```
template<class... Args>
auto capture_all(Args&&... args)
{
    return [... args = std::forward<Args>(args)]() {
        // ...
    };
}
```

Note: consteval added

```
auto pow2 = [](int n) consteval {
    return n * n;
}
```

```
constexpr int r = sqrt(5);
```

10.3 setjmp, longjmp

Note: similiar to assembler, works like goto, memorizing tag and is used for right try/catches or smth other

```
jmp_buf rp; // 128 bit number
```

```
if (setjmp(rp) == 0)
{
    foo();
}
else
{
    // ...
}
```

```
// similiar to try/catch(...) cases
```

Asserts:

```
void test_assert(bool cond; ...)
{
    if (condition) return;

    longjmp(rp; exitcode = 1);
}
```

11 Practice XI

11.1 LINQ

- Lazy execution (using during materialization)

```
String[] s = new [] { "1", "a", "b", "Ops" };
```

```
var q = s.Take(3)
        .Select(x => x + x);
```

```
var m = q.ToArray();
```

- Enumerator (iteration by 1 element next only) (forward linear iterator)

How to move strings from one collection to another:

```
std::vector<std::string> src = {"a", "b"}, dst;
std::move(src.begin(), src.end(), std::back_inserter(dst));

std::vector<int> v = {10, 20, 30, 40, 50};
auto counted = std::counted_iterator(v.begin(), 3);
for (auto& item : counted)
{
    std::cout << item; // 10, 20, 30
}
```

Tip: move iterator for faster turning set to vector (for e.g.)

11.2 ranges + homework

Def.: more effective way to optimize existing algorithms in STL

```
#include <ranges>
```

```
namespace views = std::views;
```

```
namespace ranges = std::ranges;
```

```
std::vector<int> v = {1, 2, 3, 4, 5, 6, 7};
```

```
auto r = v | views::filter([](int x) { return x % 2 == 0; })
```

```
// filter even nums
```

```
    | views::transform([](int x){ return x * x; })
```

```
// count squares
```

```
    | views::take(2);
```

```
// taking 2
```

```

for (int x : r)
{
    std::cout << x << std::endl;
}

for (auto& name : names | views::take(3))
    ...

for (auto& key : file_map | view::keys | views::reverse)
    ...

```

.Net LINQ	C++ ranges
.Where	std::views::filter
.Select	std::views::transform
.Take	std::views::take
.Skip	std::views::drop
.Reverse	std::views::reverse
.OrderBy	std::views::sort

11.3 lab

Note: tasks only during materialization

12 Practice XII

12.1 Ranges

```

std::ranges
    std::views = std::ranges::views ("fabric" of lazy adapters)

```


concepts	types that have begin() and end()
std::ranges::range	types that have begin() and end()
sized_range	Range size(), O(1)
view	Range O(1): move/copy/destruction
input_range, forward_range	
bidirectional_range	
random_access_range	
contiguous_range	
common_range	begin() and end() return same type
viewable_range	could be safely turned into view

12.2 algorithms on ranges

12.2.1 using namespace std::ranges;

all_of, any_of, none_of, count, find
search, equal, mismatch
copy, fill, transform, replace, remove
reverse
sort, stable_sort, binary_search, lower_bound
merge
includes, set_union, set_intersection
make_heap, push_heap, pop_heap
contains, starts_with, ends_with

12.2.2 major optimization improvements with ranges

```
struct Data
{
    std::string key;
    std::array<char, 256> payload;
};

std::vector<Data> vec (10000000); // a lot of data
```

```
// old approach
std::sort(vec.begin(), vec.end(), [](const Data& a, const Data& b)
        { return a.key < b.key; });

// new
std::ranges::sort(vec, std::bss {}, &Data::key);
```

12.3 concepts

```
#include <concepts>

template<typename T>
concept MyConcept = requires(T a, T b) {
    a + b;
    a - b;
    a * b;
    a / b;
    a % b;
}

concept Sortable = requires( T x ) {
    std::sort(x.begin(), x.end());
    { x.begin() } -> std::random_access_iterator;

    typename T::iterator;
    requires foo(sizeof(T));
    requires std::derived_from<T, Base>;
}

constexpr bool foo(int x)
{
    return x > 10;
}
```

```

}

template <typename T>
concept P1Funcs = requires(T x, T y)
{
    x + y;
    x - y;
}

template <typename T>
concept P2Funcs = requires(T x, T y)
{
    x * y;
    x / y;
}

template <typename T>
concept P12Funcs = !P1Funcs && P2Funcs;

// ways to use concepts

//1
template <P2Funcs T>
....

//2
template <typename T> requires P1Funcs<T>
...

//3
void foo(P1Funcs auto x) { }
template <typename T>

```

```
//4
void foo(T x) requires P1Funcs<T> { }
```

12.3.1 specialization for concepts

```
template <std::integral T>
void process(T value)
{
    ...
}
```

```
template <std::floating_point T>
void process(T value)
{
    ...
}
```

```
template <std::signed_integral T>
void process(T value)
{
    ...
}
```

If matches not only 1?

13 Practice XIII

skipped...

14 Practice XIV

14.1 concepts – how compiler understands which rule is more strict?

```
template<A T>
void foo(T x, T y) { }
```

```
template <B T>
void foo(T x, T y) { }
```

Compiler work:

1. casting A and B to normalized conjunctive form (NCF)
2. checks if $A \subseteq B$ & $B \subseteq A$
 $\Rightarrow$ xor of statements $\Rightarrow$ choose of A or B

Features:

```
concept C1 = (sizeof(T) == 4);
concept C2 = requires is_numeric<T>;

concept C3 = C1 && C2; // OK
```

```
// but C1 won't work on its own
```

```
// conflict of C1 and C2 is not solvable (on some compilers)
```

14.2 coroutines, stackful under the hood of C++20

Preferred lib: Async++ (Github: [AsyncPP](#))

14.3 C++20 modules

Compiler work: multiple handling and working with `#include` $\Rightarrow O(N \times M)$, modules $\times$ includes

First solution: Microsoft – Precompiled headers: .pch, then compiling with this file x7-10

Last sol: (for CMake ≥ 3.28) – modules (fast) added

Scheme of pipeline:

$$.pcm, .gcm \longrightarrow .o (.obj)$$

Pros: x2-10

Cons: is not version-friendly – use .cppm, .ixx

Note: modules are imported, headers are included

New key-word: export – everything within export will see statement from module with which export was used

Pros: double inclusion is prohibited.

Cons: no cyclic chains of dependencies (imports)

How to write modules?

```
export module math;
```

```
export int add( int a, int b)
{
    return a + b;
}
```

```
...
```

```
export class A { }
```

```
export typedef ...
```

```
→ .pcm
```

```
main.cpp
```

```
import math;
```

```
int main()
{
    add();

    return 0;
}
```

```
// shapes.ixx
```

```
export module shapes;
```

```
export import :circle;
```

```
export import :rectangle;
```

```
// circle.ixx — doesn't create new modules  
module shapes:circle
```

```
export class Circle { }
```

Trick for includes:

```
module;
```

```
#include <>
```

```
#include <>
```

```
...
```

```
export module legacy;
```

```
...
```

Note: some of libs from includes started to have a duplicate of its own from C++23: `import <vector>`